how changes you introduced impact your service, far beyond the traditional "all green" of testing. In the last subsection of the chapter, we will discuss how you can go about getting started in the developer observability space and what to watch out for. With that, let's jump right into the first topic: continuous profiling—what it is and when you can benefit from it.

9.1 Continuous profiling

In this subsection, we will have a look at continuous profiling, its use cases, and tooling available to you. First, we will look at its origins in the context of operating systems, and then we will review common techniques and tooling available to you. Finally, we will apply that knowledge in the context of an end-to-end walk-through example.

As usual, we will be covering open source tooling as well as commercial offerings. The open source tooling allows you to explore this space without needing to invest money for licenses or the like. After you've validated this technology, you can determine your return on investment (ROI), with it potentially paying for one of the offerings.

So what is *continuous profiling* (CP)? There are two words in this term; let's focus on the latter (profiling) first and return to what we mean by the former (continuous) later in the section. Before we get into any of the details, let's define what we mean by the term on a high level:

> *Continuous profiling enables you to understand your operating system's and services' resource usage (such as CPU or main memory and function execution times), over time. You can benefit from it in troubleshooting and performance optimization scenarios.*

You can use continuous profiling not only to understand the current performance and resource usage of your service but also to compare it with previous iterations. Think of the following use case: you add a new feature to your service and want to assess what the tradeoffs are now. On the one hand, your service is more powerful and useful now, but on the other hand, it may consume much more resources in terms of CPU or memory than you expected.

> **NOTE** In a sense, continuous profiling is related to logging, since you're operating in the context of a service, in contrast to the cross-service execution of distributed tracing. As with logging, with profiling, you're interested in certain function calls in the context of your service. However, you're interested in numerical answers, such as the time it took to execute a function or CPU utilization or the amount of main memory used in a certain code path rather than the event emitted by a log line. Continuous profiling is, from a symbols perspective (function names, data types, etc.), also related to debugging; however, in contrast to debugging, you're looking holistically at the execution, rather than stepping through single steps.

At the end of the day, whether the new functionality justifies the increased resource usage is a business decision. However, you can now, with continuous profiling, get the raw data to make that decision.